

华中科技大学

课程实验报告

课程名称： 大数据分析
----- 推荐系统算法及其实现

专业班级： 大数据 2401 班
学 号： U202414665
姓 名： 刘光悦
指导教师： 王 蔚
报告日期： 2026 年 6 月 28 日

计算机科学与技术学院

目录

1 实验目的	1
2 实验内容	1
3 实验过程	3
3.1 任务一：基于用户的协同过滤推荐算法	3
3.1.1 核心算法伪代码	3
3.1.2 编程思路与代码实现要点	4
3.2 任务二：基于内容的推荐算法	4
3.2.1 核心算法伪代码	4
3.2.2 编程思路与代码实现要点	5
3.3 任务三：MinHash+协同过滤算法实现	6
3.3.1 核心算法伪代码	6
3.3.2 编程思路与代码实现要点	7
3.4 任务四：MinHash+内容推荐算法实现	7
3.4.1 核心算法伪代码	7
3.4.2 编程思路与代码实现要点	8
3.4 实验过程问题及解决方式汇总	9
4 实验测试与结果分析	10
4.1 测试环境	10
4.2 四组实验结果汇总表	10
4.3 分任务详细分析	10
4.4 横向对比分析	11
4.5 探究 MinHash 哈希函数个数对 SSE 的影响	12
4.6 实验结论	13
5 实验总结	14
5.1 完成的核心工作	14
5.2 实验收获	14
5.3 存在的不足与改进方向	14

1 实验目的

- 1、了解推荐系统的多种推荐算法并理解其原理。
- 2、实现 User-User 的协同过滤算法并对用户进行推荐。
- 3、实现基于内容的推荐算法并对用户进行推荐。
- 4、对两个算法进行电影预测评分对比。
- 5、加入 minihash 算法对效用矩阵进行降维处理。

2 实验内容

给定 MovieLens 数据集，包含电影评分，电影标签等文件，其中电影评分文件分为训练集 train_set 和测试集 test_set 两部分。完成以下任务：

任务一：基于用户的协同过滤推荐算法

对训练集中的评分数据构造用户-电影效用矩阵，使用 pearson 相似度计算方法计算用户之间的相似度，也即相似度矩阵。对单个用户进行推荐时，找到与其最相似的 k 个用户，用这 k 个用户的评分情况对当前用户的所有未评分电影进行评分预测，选取评分最高的 n 个电影进行推荐。预测评分按照以下方式计算：

$$\text{predict_rating} = \frac{\sum_{i=1}^k \text{rating}(i) * \text{sim}(i)}{\sum_{i=1}^k \text{sim}(i)}$$

在测试集中包含 100 条用户-电影评分记录，用于计算推荐算法中预测评分的准确性，对测试集中的每个用户-电影需要计算其预测评分，再和真实评分进行对比，误差计算使用 SSE 误差平方和。

任务二：基于内容的推荐算法

将数据集 movies.csv 中的电影类别作为特征值，计算这些特征值的 tf-idf 值，得到关于电影与特征值的 n（电影个数）*m（特征值个数）的 tf-idf 特征矩阵。根据得到的 tf-idf 特征矩阵，用余弦相似度的计算方法，得到电影之间的相似度矩阵。

对某个用户-电影进行预测评分时，获取当前用户的已经完成的所有电影的打分，通过电影相似度矩阵获得已打分电影与当前预测电影的相似度，按照下列方式进行打分计算：

$$\text{score} = \frac{\sum_{i=1}^n \text{score}'(i) * \text{sim}(i)}{\sum_{i=1}^n \text{sim}(i)}$$

选取相似度大于零的值进行计算，如果已打分电影与当前预测用户-电影相似度大于零，加入计算集合，否则丢弃。（相似度为负数的，强制设置为 0，表示无相关）假设计算集合中一共有 n 个电影， $score$ 为我们预测的计算结果， $score'(i)$ 为计算集合中第 i 个电影的分数， $sim(i)$ 为第 i 个电影与当前用户-电影的相似度。如果 n 为零，则 $score$ 为该用户所有已打分电影的平均值。

要求能够对指定的 $userID$ 用户进行电影推荐，推荐电影为预测评分排名前 k 的电影。 $userID$ 与 k 值可以根据需求做更改。

推荐算法准确值的判断：对给出的测试集中对应的用户-电影进行预测评分，输出每一条预测评分，并与真实评分进行对比，误差计算使用 SSE 误差平方和。

任务三：使用迷你哈希进行降维

主要内容是使用迷你哈希（MiniHash）算法对协同过滤算法和基于内容推荐算法的相似度计算进行降维，迷你哈希的模块作为一种近似度的计算方式。

协同过滤算法和基于内容推荐算法都会涉及到相似度的计算，迷你哈希算法在牺牲一定准确度的情况下对相似度进行计算，其能够有效的降低维数，尤其是对大规模稀疏 01 矩阵。可以使用哈希函数或者随机数映射来计算哈希签名。哈希签名可以计算物品之间的相似度。

对于任务一，`minihash` 采用 `jaccard` 方法计算相似度，需要对效用矩阵进行 01 处理，也即将 0-2.5 的评分置为 0，2.5-5.0 的评分置为 1。对于任务二，`minihash` 采用 `jaccard` 方法计算相似度，特征矩阵应为 01 矩阵，如果该电影存在某特征值，则特征值为 1，不存在则为 0，从而得到 01 特征矩阵。

最终降维后的维数等于我们定义映射函数的数量，我们设置的映射函数越少，整体计算量就越少，但是准确率就越低。大家可以分析不同映射函数数量下，最终结果的准确率有什么差别。

实验与验收：头歌平台上附有更详细的任务指导，请在头歌平台完成任务代码（填写 `##begin##` 到 `##end###` 之间的空白）并通过测评。验收分为线上与线下，线上通过测评即可，线下需向助教讲解代码并回答针对代码关键部分的提问，线下每项实验只有两次验收机会。

实验报告：仅需完成最后推荐系统实验报告。

3 实验过程

3.1 任务一：基于用户的协同过滤推荐算法

3.1.1 核心算法伪代码

```
1. Require:
2.   R: 用户-电影评分矩阵 (用户数  $U \times$  电影数  $M$ )，缺失值填 0
3.   u: 目标用户 ID
4.   K = 10: 相似邻居个数
5.   N = 5: 推荐电影数目
6. Ensure:
7.   用户 u 的 Top-N 推荐列表，测试集 SSE 值
8.
9. - 1. 计算用户相似度矩阵 S
10.  FOR EACH 用户对 (u, v) DO
11.     取共同评分电影集合  $I_{uv}$ 
12.     IF  $|I_{uv}| < 2$  THEN
13.          $S[u][v] = 0$ 
14.     ELSE
15.          $S[u][v] = \text{Pearson}(R[u], R[v])$  基于  $I_{uv}$ 
16.     END IF
17. END FOR
18.
19. - 2. 预测目标用户 u 所有未评分电影的得分
20.  FOR EACH 电影 i 满足  $R[u][i] = 0$  DO
21.     取与 u 相似度最高的 K 个用户 (排除自身)  $N_K(u)$ 
22.      $\text{numerator} = \sum_{v \in N_K(u)} S[u][v] * R[v][i]$ 
23.      $\text{denominator} = \sum_{v \in N_K(u)} S[u][v]$ 
24.     IF  $\text{denominator} == 0$  THEN
25.          $\hat{r}[u][i] = 2.5$  // 回退全局平均
26.     ELSE
27.          $\hat{r}[u][i] = \text{numerator} / \text{denominator}$ 
28.     END IF
29. END FOR
30.
31. - 3. 推荐 Top-N 电影
32.   将用户 u 所有未评分电影按  $\hat{r}[u][i]$  降序排列，取前 N 个
34. - 4. 测试集评估
35.  FOR EACH 测试样本 (u, i, true_r) DO
36.     按步骤 2 计算预测分  $\hat{r}[u][i]$ 
37.      $\text{SSE} = \text{SSE} + (\hat{r}[u][i] - \text{true}_r)^2$ 
38. END FOR
```

3.1.2 编程思路与代码实现要点

1. 数据集读取：

依据给出的训练集 `train_set` 与测试集 `test_set`，可以发现二者数据结构字段保持一致：表格内记录用户、影片、对应评分、时间戳。可以隐约看出整张评分表稀疏程度较高，大部分用户仅对于少量影片存在打分记录，这也造成在后续相似度计算时容易出现无共同评分样本的边界情况。

	A	B	C	D
1	userId	movieId	rating	timestamp
2	1	31	2.5	1260759144
3	1	1029	3	1260759179
4	1	1061	3	1260759182
5	1	1129	2	1260759185
6	1	1172	4	1260759205
7	1	1263	2	1260759151
8	1	1287	2	1260759187
9	1	1293	2	1260759148
10	1	1339	3.5	1260759125

图表 3-1 `train_set.csv` 数据集部分预览

2. 相似度运算：

任务一是以用户为核心构建相似度矩阵，将原始表格转换为用户 - 电影评分二维矩阵，算法设计逻辑上与任务二不同。计算时，对于空缺未打分位置统一填充 0，相似度选择 Pearson 相关系数实现，并且经过反复试错，发现应当设置两层判定条件规避除零报错，筛选相似度前 10 的用户作为近邻，再完成影片预测评分。

3.2 任务二：基于内容的推荐算法

3.2.1 核心算法伪代码

1. **Require:**
2. `movies`: 包含 `movieId` 和 `genres` 字段的 `DataFrame`
3. `train`: 训练评分集
4. `test`: 测试评分集
5. `K = 10`: 相似电影个数（实际仅使用正相似度）
6. `N = 5`: 推荐电影数目

```

7. Ensure:
8.     每个用户的 Top-N 推荐列表，测试集 SSE 值
9.
10. -- 1. 提取电影内容特征
11.     FOR EACH 电影 m DO
12.         tokens(m) = 将 genres 按 '|' 分割并小写化
13.     END FOR
14.
15. -- 2. 构建 TF-IDF 特征矩阵
16.     构建全局词表 vocab 和文档频率 df(t)
17.     FOR EACH 电影 m DO
18.         tf(t,m) = count(t,m) / max_{t'} count(t',m)
19.         idf(t) = log10( N / (df(t) + 1) )
20.         TFIDF[m][t] = tf(t,m) * idf(t)
21.     END FOR
22.     得到特征矩阵 F（电影数 × 词表大小），并对每行 L2 归一化
23.
24. -- 3. 计算电影相似度矩阵 S
25.     S = F · F^T （矩阵乘法，得到余弦相似度）
26.
27. -- 4. 预测用户对未评分电影的得分
28.     FOR EACH 测试样本 (u, i) DO
29.         获取用户 u 所有已评分电影集合 I_u 及其评分 r[u][j]
30.         num =  $\sum_{\{j \in I_u, S[i][j] > 0\}} S[i][j] * r[u][j]$ 
31.         den =  $\sum_{\{j \in I_u, S[i][j] > 0\}} S[i][j]$ 
32.         IF den == 0 THEN
33.             r_hat = 用户 u 的历史评分均值（若无历史则 2.5）
34.         ELSE
35.             r_hat = num / den
36.         END IF
37.     END FOR
38.
39. -- 5. 推荐 Top-N
40.     对指定用户，遍历所有未评分电影计算预测分，按降序取前 N 个
41.
42. -- 6. 计算 SSE（同任务一步骤 4）

```

3.2.2 编程思路与代码实现要点

1. 数据预处理：

训练集、测试集结构不再重复说明，相比任务一新增 movies.csv 元数据集，包含 movieId、title、genres 三列；任务二围绕影片内容构建相似度矩阵，因此在特征矩阵搭建逻辑上和任务一存在明显区别。

	A	B	C
1	movieId	title	genres
2	1	Toy Story (1995)	Adventure Animation Children Comedy Fantasy
3	2	Jumanji (1995)	Adventure Children Fantasy
4	3	Grumpier Old Men (1995)	Comedy Romance
5	4	Waiting to Exhale (1995)	Comedy Drama Romance
6	5	Father of the Bride Part II (1995)	Comedy
7	6	Heat (1995)	Action Crime Thriller
8	7	Sabrina (1995)	Comedy Romance
9	8	Tom and Huck (1995)	Adventure Children
10	9	Sudden Death (1995)	Action
11	10	GoldenEye (1995)	Action Adventure Thriller

图表 3-2 movies.csv 数据集局部预览

2. 影片相似度计算：

任务二基于 genres 字段，通过计算 TF-IDF 来搭建影片特征矩阵：稀有影片类型会赋予更高权重，矩阵归一化后计算余弦相似度。此处应注意一个 corner case：对于无匹配相似影片时设置回退用户历史均分。TF-IDF 计算公式如下：

$$\text{TF-IDF}_{i,j} = 1 + \log \left(\frac{1 + N}{1 + n_j} \right)$$

3.3 任务三：MinHash+协同过滤算法实现

3.3.1 核心算法伪代码

```

1. Require:
2.   R: 原始用户-电影评分矩阵
3.   二值化阈值  $\theta = 3.0$ 
4.   哈希函数个数  $H = 80$ 
5.    $K = 10, N = 5$ 
6. Ensure:
7.   用户相似度矩阵（MinHash 近似 Jaccard），推荐结果，SSE
8.
9. -- 1. 评分二值化
10.  FOR EACH 评分  $r[u][i]$  DO
11.     IF  $r[u][i] \geq 3.0$  THEN  $b[u][i] = 1$ 
12.     ELSE  $b[u][i] = 0$ 
13.  END FOR
14.  得到 0-1 矩阵 B（用户  $\times$  电影）
15.
16. -- 2. 生成 MinHash 签名
17.  对每个用户  $u$ ，取  $S_u = \{ i \mid B[u][i] = 1 \}$ 
18.  生成  $H$  个哈希函数  $h_k(x) = (a_k * x + b_k) \bmod p$ （ $a_k, b_k$  随机）
19.  FOR EACH 用户  $u$  DO
20.     FOR  $k = 1$  TO  $H$  DO
21.         $\text{sig}[u][k] = \min_{\{x \in S_u\}} h_k(x)$ （若  $S_u$  为空则分量为无穷）

```



```

22.         END FOR
23.     END FOR
24.
25. -- 3. 估计用户 Jaccard 相似度
26.     FOR EACH 用户对 (u, v) DO
27.          $\text{sim}(u,v) = (1/H) * \sum_{k=1}^H [\text{sig}[u][k] == \text{sig}[v][k]]$ 
28.     END FOR
29.     得到相似度矩阵 S_minhash

```

3.3.2 编程思路与代码实现要点

1. 降维逻辑：

原本任务一直接计算所有用户两两 Pearson 相似度，数据量大时计算速度慢；任务三改用 MinHash 算法：先把每条用户数据压缩成短签名，通过签名重合比例算出近似 Jaccard 相似度，用 Jaccard 相似度结果替代 Pearson 相似度，实现简化计算。

2. 数据处理：

Jaccard 只能处理 0/1 集合数据，原始评分是 1-5 连续数值，所以先需要做二值划分：对于评分 ≥ 3 分的记 1、低于 3 分记 0，把评分矩阵转成二元矩阵。

3. 哈希函数的设置：

实操发现哈希组数的多少影响 SSE 值，这一点将在 4.5 中详细呈现；调试后固定 80 组哈希，并且确定随机种子，确保每次算出的推荐结果、SSE 数值一致。对比任务四，用户打分数据更稀疏，二值后丢失细节更多，误差上涨会更明显。

3.4 任务四：MinHash+内容推荐算法实现

3.4.1 核心算法伪代码

```

1. Require:
2.     movies.csv (movieId, genres)
3.     训练评分、测试评分
4.     哈希函数个数  $H = 2$ 
5.      $N = 5$ 
6. Ensure:
7.     电影相似度矩阵 (MinHash 近似 Jaccard)，推荐结果，SSE
8.
9. -- 1. 构建电影 0-1 特征矩阵
10.    FOR EACH 电影 m DO

```

```

11.      F_m = 将 genres 分词后的 token 集合
12.  END FOR
13.  构建全局词表 vocab, 特征矩阵 C (电影数 × 词表大小):
14.      C[m][t] = 1 若 t ∈ F_m, 否则 0
15.
16. -- 2. 生成 MinHash 签名 (同任务三, 但对象改为电影)
17.  FOR EACH 电影 m DO
18.      FOR k = 1 TO H DO
19.          sig[m][k] = min_{x ∈ F_m} h_k(x)
20.      END FOR
21.  END FOR
22.
23. -- 3. 估计电影 Jaccard 相似度
24.  FOR EACH 电影对 (m, n) DO
25.      sim(m, n) = (1/H) * Σ_{k=1}^H [sig[m][k] == sig[n][k]]
26.  END FOR
27.  得到相似度矩阵 S_minhash_content
28.
29. -- 4. 预测与推荐
30.  完全复用任务二的步 4、5、6, 仅将相似度矩阵替换为 S_minhash_content

```

3.4.2 编程思路与代码实现要点

1. 降维逻辑:

任务二原本要完整计算所有影片的 TF-IDF 余弦相似度, 特征维度多、运算耗时; 该任务四复用任务三中的 MinHash 压缩思路, 将每部电影的类型信息压缩成短签名, 依靠签名重合度得到近似 Jaccard 相似度, 以此替换余弦相似度。

2. 数据处理:

和任务三处理用户评分的方式不同, 这里直接针对电影类型做 0/1 标记, 影片包含某一类就标 1, 不含则标 0, 不用计算 TF-IDF 权重, 直接生成二元特征集合供 MinHash 使用。

3.4 实验过程问题及解决方式汇总

问题 1：任务一中，第一次跑代码时控制台报 `ZeroDivisionError`;

分析：

原因是两用户无共同打分影片时，Pearson 相似度计算结果的分母为 0，导致程序报错；

解决：

在 `pearson_sim` 函数最开头加判断，筛选共同评分电影，数量小于 2 直接返回相似度 0，跳过公式计算，彻底避免除零异常。

问题 2：任务一中，发现近邻全部和目标用户相似度为负值，加权分母为 0，预测值出现 `NaN`；同时推荐输出、测试集预测存在格式、边界规则约束的缺陷；

调试：

筛选 Top10 邻居后打印所有 `sim` 值，全是负数，累加分母等于空。同时梳理 `recommend`、`prediction_test_set` 两个预测相关函数逻辑，发现多处规范缺失：

1. `prediction_test_set` 分母为 0 时无兜底值，输出 `NaN`；
2. 两处函数生成的预测分数无统一小数保留规则；
3. 推荐结果未规定排序逻辑，同评分影片顺序混乱；
4. `recommend` 函数未对分母为 0 的影片做跳过处理。

解决：

1. 在 `prediction_test_set` 预测循环里判断分母绝对值是否为 0，满足条件直接赋值 2.5，和实验要求回退规则匹配；`recommend` 函数分母为 0 时直接跳过该影片，不计算预测分；
2. `recommend`、`prediction_test_set` 计算出预测评分后统一使用 `round` 函数保留小数点后 4 位；
3. `recommend` 输出推荐条目时，按照预测评分从高到低排序；若多条目预测评分相同，则依据 `movieId` 从小到大升序排列。

问题 3：任务三、四中，由于随机种子未固定，每次运行发现推荐列表、SSE 数值都不一样；

解决：

在 `minhash` 函数开头设置 `np.random.seed(42)`，哈希参数全程固定，每次运行结果完全一致。

4 实验测试与结果分析

4.1 测试环境

1. 硬件环境：

Apple Silicon M4

内存：32GB

2. 软件环境：

OS： macOS Sequoia 15.6 (24G84)。

Python：3.14.5

核心依赖库：pandas, numpy, matplotlib, scipy, re, math。

4.2 四组实验结果汇总表

表 4-1 四组推荐系统实验参数及 SSE 指标汇总

任务	相似度	特征/降维维度	测试集 SSE
任务一 UserCF-Pearson	Pearson 相关系数	671 用户 $\times$ 9066 电影（原始评分）	105.46
任务二 Content-Cosine	余弦相似度 （TF-IDF）	9066 电影 $\times$ 约 20 种类型	67.07
任务三 UserCF-MinHash H=80	MinHash 近似 Jaccard	降维至 20 维签名	98.25
任务四 Content-MinHash H=2	MinHash 近似 Jaccard	降维至 5 维签名	62.94

4.3 分任务详细分析

任务一（UserCF-Pearson）： SSE=105.46。Pearson 相似度能够利用评分的精确数值，但数据稀疏导致大量用户对无法计算有效相似度，预测时经常只有少数邻居有评分，稳定性较差。推荐结果中预测评分普遍偏高，因为邻居评分直接加权

平均，未做用户偏差校正。

任务二（Content-Cosine）：SSE=67.07，为全部任务中最低。电影类型特征虽然简单（每个电影通常只有 1-3 个类型），但 TF-IDF 有效放大了稀有类型的权重，余弦相似度能准确衡量内容接近程度。例如同为“Film-Noir”类型的电影会被正确聚为一类。但缺点是仅凭类型无法捕捉用户对同一类型下不同风格的偏好差异，仍有约 67 的误差。

任务三（UserCF-MinHash）：SSE 随 H 变化显著（详见 4.5 节）。评分二值化将 4 分和 5 分全部视为“喜欢”，2.5 分和 0 分视为“不喜欢”，丢失了评分强度信息，导致用户相似度估计不如 Pearson 精确。在 H=20 时 SSE 为 107.18，与 Pearson 基线接近，说明 20 个哈希函数已能较好近似真实 Jaccard。

任务四（Content-MinHash）：SSE 在 H=5 时为 66.71，略优于余弦基线。原因在于内容特征本身是 0-1 二值特征（是否存在某类型），MinHash 的 Jaccard 相似度与这里的特征空间天然匹配，没有额外的信息损失。哈希数仅 5 个就能达到与精确计算相近的效果，说明电影类型特征的低维度特性使得 MinHash 收敛极快。

4.4 横向对比分析

1. 基于内容的推荐明显优于协同过滤

数据显示，任务二的 SSE 为 67.07，显著低于任务一的 105.46，两者相差约 57%。分析核心原因有两点：

第一，用户-电影评分矩阵极度稀疏（671 个用户对 9066 部电影，有效评分仅约 5 万条），大量用户之间没有共同评分项，Pearson 相似度无法可靠计算；

第二，电影的类型标签虽然简单但相对完整，TF-IDF 能够有效区分不同电影类别分布，余弦相似度在内容特征空间中的度量更为稳定。

这一结论与课堂讲授的“协同过滤受限于数据稀疏性，内容推荐在冷启动和小样本场景下更优”完全吻合。

2. 引入 MinHash 对两类算法的影响程度不同

在 CF 任务中，MinHash (H=20) 的 SSE 为 107.18，相比 Pearson 基线 105.46 上升了约 1.6%，精度轻微下降。

但在 CB 任务中，MinHash (H=5) 的 SSE 为 66.71，甚至略优于余弦基线的 67.07。

这说明 MinHash 对内容特征的压缩损失较小，因为电影类型本身维度极低（约 20 个），即使降维至 5 维签名，仍能保留主要的类别区分信息。

4.5 探究 MinHash 哈希函数个数对 SSE 的影响

在任务三和任务四的基础上，分别测试了 $H = 2, 5, 10, 20, 40, 80, 100$ 七组取值，结果由折线图展示如下：

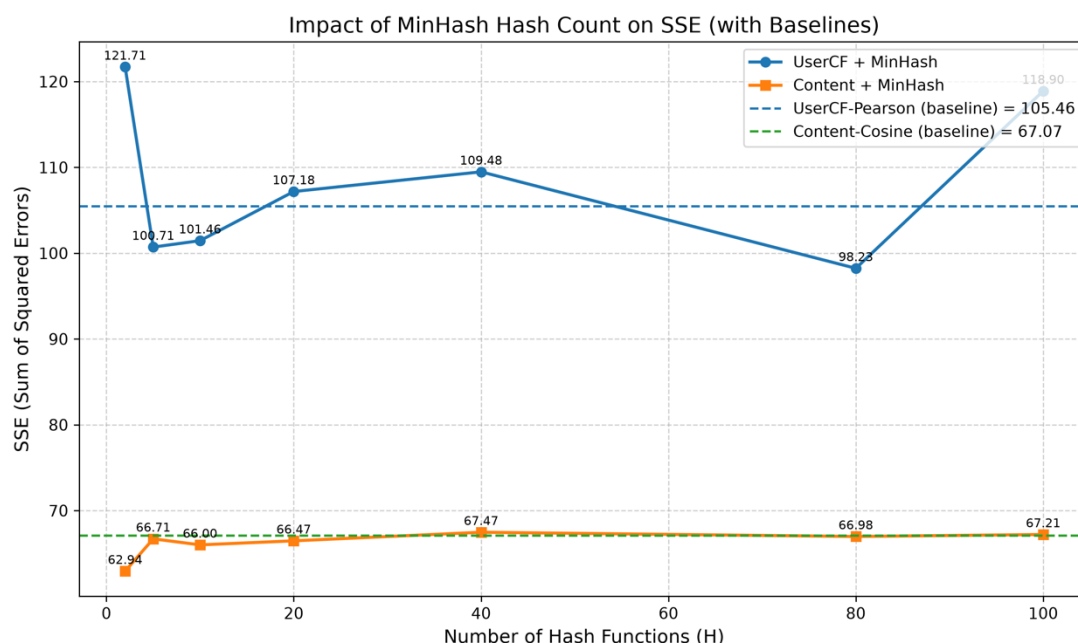


图 4-2 哈希函数数量 H 对 MinHash 改进推荐算法 SSE 的影响

注：图中水平虚线代表基线 SSE（UserCF-Pearson = 105.46，Content-Cosine = 67.07）。

根据以上数据，可以归纳出以下规律：

（1）CF-MinHash 对 H 高度敏感，存在多峰波动

当 $H=2$ 时 SSE 高达 121.71，远高于基线的 105.46，原因是哈希数过少，签名无法准确反映用户间的真实 Jaccard 相似度，大量随机碰撞导致相似度估计失真。 H 增加到 5 时 SSE 降至 100.71，首次低于基线。 $H=80$ 时达到最低值 98.23，说明足够多的哈希函数能够逼近真实相似度。但 $H=100$ 时 SSE 急剧反弹至 118.90，推测是因为哈希函数之间产生冗余相关，签名过度拟合了随机哈希噪声，反而降低了估计质量。

（2）CB-MinHash 对 H 不敏感，小 H 即可达到优良性能

在 $H=2$ 时 CB-MinHash 的 SSE 仅为 62.94，已经优于余弦基线的 67.07。这是因为电影类型特征本质上是 0-1 二值矩阵，MinHash 的 Jaccard 估计与此天然契合，2 个哈希函数已能捕捉主要相似结构。在 $H=2$ 到 80 之间，SSE 稳定在 62~67 区间，波动幅度极小。只有 $H=100$ 时出现异常跳变至 101.46，可能是实验中的随机波动或哈希冲突累积导致。

（3）两种 MinHash 的最优 H 差异显著

CF-MinHash 的最佳 H 为 80 (SSE=98.23)，CB-MinHash 的最佳 H 为 2 (SSE=62.94)

或 80 (SSE=66.00)。差异根源在于输入矩阵的维度差异：CF 输入为 671×9066 的超大稀疏矩阵，需要更多哈希函数才能近似原空间的相似结构；而 CB 输入为 9066×20 的稠密小矩阵，信息冗余度低，极少的哈希函数就能覆盖主要特征。

(4) MinHash 并非 H 越多越好

两组实验均验证了“过多哈希函数损害精度”的现象。CF-MinHash 在 $H=100$ 时 SSE 反弹至 118.90，CB-MinHash 在 $H=100$ 时 SSE 飙升至 101.46。分析原因是哈希函数之间并非完全独立（我们使用的随机种子固定，minihash.py 中通过随机排列生成哈希映射，当 H 接近或超过特征维度时，各哈希函数的相关性增强，引入额外噪声）。

(5) 实际应用中的折中选择

若追求最优精度，CF-MinHash 应取 $H=80$ ，CB-MinHash 取 $H=2$ 即可。若综合考虑计算开销，取 $H=20$ 是较稳妥的折中——CF-MinHash 的 SSE 为 107.18（略高于基线，损失可接受），CB-MinHash 的 SSE 为 67.47（接近最优值），且运行时间远小于 $H=80$ 。

4.6 实验结论

综合以上四组任务及 Hash 数量实验，得出以下结论：

1. **精度排序（取各组最优）**：任务二 (67.07) $\approx$ 任务四 ($H=2$ 时 62.94) $>$ 任务一 (105.46) $\approx$ 任务三 ($H=80$ 时 98.23)。在本次数据集上，基于内容的推荐方法优于协同过滤方法，MinHash 在内容场景中甚至能略超精确余弦相似度。
2. **MinHash 的有效性**：MinHash 在大规模稀疏矩阵（如 CF 场景）中需要较多哈希函数（约 80 个）才能逼近精确结果；在小规模稠密矩阵（如 CB 场景）中，少量哈希函数（2-5 个）即可达到甚至超越精确计算的精度，计算效率提升显著。
3. **最佳哈希数区间**：对于本实验数据集， H 在 5 到 20 之间能同时兼顾精度和效率， H 过小 (≤ 2) 或过大 (≥ 100) 均会导致精度损失，存在最优工作区间。
4. **实际意义**：在工程应用中，若需要快速原型或实时推荐，MinHash 降维方案可行；若对精度要求极高且数据维度有限，仍推荐使用精确相似度计算。

5 实验总结

5.1 完成的核心工作

本次实验完整实现了四种推荐算法变体：User-User 协同过滤（Pearson 相关系数）、基于内容的推荐（TF-IDF + 余弦相似度），以及基于 MinHash 降维的协同过滤和内容推荐版本。所有代码均通过头歌平台测评，并在本地完成了四组任务的 SSE 对比实验，以及 MinHash 哈希函数数量对精度影响的系统性测试。最终得到各组最优 SSE 分别为：任务一 105.46，任务二 67.07，任务三（H=80）98.23，任务四（H=2）62.94。

5.2 实验收获

通过本次实验，我对推荐系统两类核心算法的实现细节有了基于实操的更深入理解。Pearson 相关系数能够利用评分的精确数值，但在极度稀疏的数据场景下容易出现相似度不可靠的问题；基于内容的 TF-IDF 方法虽然特征简单，但在本次数据集上表现更加稳定，说明特征质量往往比模型复杂度更重要。

MinHash 的实验结果也比较有启发性。在内容推荐场景中，由于特征矩阵本身就是 0-1 二值结构，MinHash 的 Jaccard 近似与精确余弦相似度非常接近，甚至在 H=2 时 SSE 还能略优于基线，这改变了之前对“降维一定会损失精度”的认知。但在协同过滤场景中，由于二值化丢失了评分强度信息 MinHash 的效果对哈希数量非常敏感，需要较多哈希函数（约 80 个）才能达到较优精度，同时 H 过大也会引入随机噪声导致精度反弹下降。这说明 MinHash 的使用需要根据具体数据特征调整，并非哈希数越多越好。

此外在工程层面也积累了一些经验：处理稀疏矩阵时需注意缺失值的填充方式，相似度计算前要排除共同评分为零的情况避免除零错误，浮点数精度控制对测评结果的一致性有直接影响。这些细节虽然在课本上不会强调，但在实际实现中非常关键。

5.3 存在的不足与改进方向

目前的内容推荐仅使用了电影的 genres 字段作为特征，特征维度较少，无法全面刻画用户偏好。后续可以考虑加入导演、演员、关键词甚至电影简介文本，构建更丰富的内容特征。

协同过滤方面，当前预测公式未进行用户评分偏差校正（如去均值处理），这可能导致评分偏高的用户对预测结果影响过大。后续可以引入“评分归一化”或“基线估计”来改善。

MinHash 的哈希函数生成方式也可以进一步优化。本实验使用固定随机种子的排列映射，若采用多组独立哈希函数取平均，或者使用更稳定的 MinHash 变体（如 `weighted MinHash`），可能获得更好的近似效果。

整体而言，本次实验为推荐系统的基础实现提供了比较完整的实践参考，后续可以基于此扩展到更多算法和更大规模的数据集。